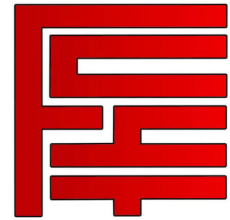


**UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
CARRERA DE INGENIERÍA DE SISTEMAS**



**SISTEMA INTELIGENTE DE ENTRENAMIENTO
EN GIMNASIOS CON REALIDAD AUMENTADA E
INTELIGENCIA ARTIFICIAL**

Proyecto de Grado presentado para optar al Diploma Académico de
Licenciatura en Ingeniería de Sistemas

Presentado por: Erika Chino Alanoca

Tutor: Mgr. Jimmy Villarroel Novillo

Cochabamba – Bolivia

2026

AGRADECIMIENTOS

[**PENDIENTE:** Escribir los agradecimientos.]

FICHA RESUMEN

Grado:	Licenciatura en Ingeniería de Sistemas
Modalidad:	Proyecto de Grado
Facultad:	Ciencias y Tecnología
Carrera:	Ingeniería de Sistemas
Título del trabajo:	Sistema Inteligente de Entrenamiento en Gimnasios con Realidad Aumentada e Inteligencia Artificial
Autora:	Erika Chino Alanoca
Tutor:	Mgr. Jimmy Villarroel Novillo

En el primer capítulo se realizó una investigación sobre los antecedentes y se analizó la problemática relacionada con el abandono temprano de los usuarios de gimnasios, atribuida principalmente a la falta de orientación, motivación y seguimiento personalizado. A partir de ello, se propone el diseño de un sistema inteligente capaz de guiar al usuario en uso correcto de máquinas, personalizar rutinas de entrenamiento y registrar su progreso físico.

En el segundo capítulo se desarrolla el marco teórico que sustenta la solución tecnológica. Se revisan distintas metodologías de desarrollo de software, comparando enfoques tradicionales y ágiles, y se justifica la selección de Mobile-D como la metodología más adecuada para este proyecto.

En el tercer capítulo se detalla el área de aplicación y se describe la adopción de la metodología Mobile-D en el desarrollo del sistema, especificando su aplicación práctica en fases de exploración, inicialización, producción, estabilización y pruebas.

ÍNDICE GENERAL

I	INTRODUCCIÓN	1
1.1	Antecedentes	1
1.2	Análisis del problema	2
1.2.1	Descripción del problema	2
1.2.2	Definición del problema	3
1.3	Objetivos	3
1.3.1	Objetivo general	3
1.3.2	Objetivos específicos	3
1.4	Área de conocimiento	4
1.5	Alcance	4
1.6	Justificación	4
1.6.1	Justificación técnica	4
1.6.2	Justificación económica	5
1.6.3	Justificación social	5
II	MARCO TEÓRICO	6
2.1	Metodología de Desarrollo	6
2.2	Metodologías de desarrollo de software tradicionales vs ágiles	6
2.2.1	Metodologías tradicionales	6
2.2.2	Metodologías ágiles para móvil	8
2.3	Mobile-D	10
2.3.1	Fases y etapas	11
2.4	Programación Móvil	12
2.5	Sistema Operativo	14
2.5.1	Android	14
2.5.2	Arquitectura de Android	14
2.5.3	Ventajas y limitaciones	15
2.6	Realidad Aumentada	15
2.6.1	Componentes de un sistema de Realidad Aumentada	16

2.6.2	Tecnologías para el desarrollo de Realidad aumentada móvil	16
2.7	Inteligencia Artificial	17
2.7.1	Aprendizaje automático	17
2.7.2	Redes neuronales	17
2.7.3	Visión por computadora	18
2.7.4	Modelos de lenguaje Extenso (LLM)	18
2.8	Fundamentos del entrenamiento físico	18
2.8.1	Entrenamiento físico	18
2.8.2	Principios del entrenamiento físico	19
2.8.3	Biomecánica básica	19
2.8.4	Lesiones comunes por mala técnica	19
2.8.5	Máquinas de gimnasio	20
2.9	Plataforma de desarrollo	21
2.9.1	Visual Studio Code	21
2.10	Frameworks para desarrollo Mobile	22
2.10.1	React Native	24
2.11	Bases de datos de aplicaciones Móviles	25
2.11.1	Firebase	27
2.12	Modelos de Lenguaje Extenso (LLM)	28
2.12.1	OpenAI	28
2.13	Herramientas	29
2.13.1	Control de versiones	29
2.13.2	Herramientas de Gestión	29
III	ÁREA DE APLICACIÓN	31
3.1	Antecedentes	31
3.1.1	Enfoque metodológico	31
3.2	Metodología de desarrollo	31
3.3	Aplicación de la Metodología Mobile-D	32
3.3.1	Fase de Exploración	32
3.3.2	Fase de Inicialización	34
3.3.3	Fase de Producción	36
3.3.4	Fase de Estabilización	38
3.3.5	Fase de Pruebas del sistema	38
3.4	Roles	39
3.5	Técnicas e instrumentos de recolección de información	40

IV	DESARROLLO DEL PROYECTO	41
4.1	Fase de Exploración	41
	BIBLIOGRAFÍA	42

ÍNDICE DE TABLAS

2-1	Comparación de Metodologías tradicionales de desarrollo de software . . .	7
2-2	Comparación de metodologías ágiles móviles	9
2-3	Comparación de Android vs iOS	13
2-4	Comparación de frameworks de desarrollo Mobile	23
2-5	Comparación de bases de datos para aplicaciones móviles	26
2-6	Comparación de modelos de lenguaje extenso	28
3-1	Roles establecidos por Mobile-D	40

ÍNDICE DE FIGURAS

2-1	Fases y etapas de la metodología Mobile-D (Leyva, León, & García, 2016)	11
2-2	Arquitectura de Android (Gómez A. A., 2015)	15
2-3	Interfaz de Visual Studio Code	22

CAPÍTULO I: INTRODUCCIÓN

En la actualidad, la práctica del ejercicio físico en gimnasios ha cobrado gran relevancia como parte de un estilo de vida saludable. Sin embargo, diversos estudios han evidenciado que una gran parte de los usuarios abandonan sus rutinas en los primeros meses, principalmente por la falta de orientación, motivación y seguimiento personalizado (Gjestvang, 2020). Esta problemática afecta especialmente a los principiantes, quienes enfrentan barreras como el desconocimiento del uso correcto de las máquinas, el alto coste de entrenadores personales y la ausencia de planificación estructurada.

En respuesta a esta necesidad, se han desarrollado diversas soluciones tecnológicas que buscan mejorar la experiencia del usuario en el gimnasio, como aplicaciones móviles con realidad aumentada (Cruz, 2024) y sistemas de información para rutinas y dietas (Fernández, 2018). No obstante, muchas de estas herramientas carecen de personalización profunda y seguimiento inteligente.

Este proyecto propone el desarrollo de una aplicación móvil basada en realidad aumentada e inteligencia artificial, orientada a guiar a los usuarios en el uso correcto de máquinas de ejercicio, planificar rutinas personalizadas y realizar un seguimiento del progreso físico. La solución busca democratizar el acceso a la orientación profesional, reducir el abandono temprano y fomentar la constancia en el entrenamiento físico, utilizando tecnologías modernas como React Native, Firebase y TensorFlow Lite.

Con esto, se pretende contribuir no solo al bienestar individual, sino también a la optimización de recursos en gimnasios, ofreciendo una herramienta accesible, interactiva y eficaz para la autogestión del entrenamiento.

1.1. Antecedentes

(Yépez, 2017) en su proyecto de grado cuyo objetivo principal es la creación de un gimnasio que combine asesoramiento presencial y virtual, orientado a la nueva disciplina llamada Street Workout, que consiste en utilizar el propio peso corporal para ejercitarse.

(Gjestvang, 2020) se realizó un estudio longitudinal en Noruega con el objetivo de identificar las principales barreras en la iniciación y adherencia al ejercicio en gimnasios. El proyecto evidenció que entre el 40 % y el 65 % de los nuevos usuarios abandonan la asistencia regular en los primeros meses, principalmente por la falta de motivación, orientación

adecuada y seguimiento personalizado. Estos resultados destacan la importancia de diseñar herramientas tecnológicas que fortalezcan la adherencia a programas de ejercicio estructurados.

(Cruz, 2024) desarrolló un proyecto de grado desarrollado en la Universidad Mayor de San Simón, una aplicación móvil para gimnasios que utiliza escaneo de código QR con realidad aumentada para mejorar la experiencia de los usuarios y la gestión de entrenamientos, que permite a los usuarios obtener instrucciones detalladas en tiempo real sobre los ejercicios.

(Fernandez, 2018) en su proyecto de grado desarrolló un sistema de información cuyo objetivo principal es brindar apoyo a los usuarios o clientes de los gimnasios, proporcionándoles rutinas de ejercicios y dietas, para que sirva de guía sobre cómo usar las máquinas y como administrar sus alimentos para obtener el objetivo deseado.

(BodBot IA Entrenador Personal, 2016) es una plataforma de fitness avanzada creada por Edward Laux, Jason Storm y Sergio Prado, que ofrece entrenamientos de IA personalizados según los objetivos, equipos, capacidades físicas y dificultad del usuario, esta plataforma proporciona una guía paso a paso para cada ejercicio, contiene personalización de peso corporal y registro de alimentación diario.

(Ejercicios en el Gimnasio, 2021) desarrollada por Leap Fitness Group, es una plataforma fitness para principiantes y asiduos al gimnasio, que funciona como un entrenador personalizado, creando rutinas, mostrando la postura correcta para realizar ejercicios esto por medio de instrucciones visuales y literales explícitas, siguiendo el progreso con estadísticas y gráficos claros.

1.2. Análisis del problema

1.2.1. Descripción del problema

Este proyecto surge para resolver las principales barreras que los usuarios de gimnasios enfrentan, particularmente la falta de orientación adecuada y la falta de estructura en sus entrenamientos. (Gjestvang, 2020) Estudios demuestran que entre el 40 % y el 65 % de los nuevos usuarios abandonan el gimnasio en los primeros meses, principalmente por la falta de guía y seguimiento personalizado. Los principales problemas identificados incluyen:

- **Falta de conocimiento sobre el uso adecuado de las máquinas y la postura correcta:** Esto lleva a errores que pueden causar lesiones y dificultar el progreso hacia los objetivos físicos.

- **Costo elevado de los entrenadores personales:** Muchas personas no pueden acceder a una guía profesional debido al costo asociado, lo cual deja a la mayoría sin un apoyo adecuado.
- **Falta de seguimiento del progreso:** La falta de un sistema para registrar y analizar el progreso personal desmotiva a los usuarios, ya que no tienen una forma clara de visualizar sus avances.
- **Carencia de planificación estructurada del entrenamiento:** Sin una rutina clara y bien planificada, muchos usuarios se sienten perdidos y no logran aprovechar el tiempo de entrenamiento, lo cual fomenta la desmotivación y el abandono.

1.2.2. Definición del problema

¿Cómo desarrollar una aplicación para gimnasios que ofrezca seguimiento rutinas de ejercicio personalizadas con Inteligencia artificial?

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un sistema inteligente para el entrenamiento en gimnasios, proporcionando una guía estructurada y seguimiento personalizado de los ejercicios.

1.3.2. Objetivos específicos

- Guiar sobre el uso correcto de máquinas.
- Instruir en la ejecución correcta de ejercicios específicos.
- Diseñar rutinas de entrenamiento personalizadas y estructuradas, adaptadas a los objetivos de cada usuario.
- Construir herramientas de planificación que permitan a los usuarios organizar sus entrenamientos de manera eficiente.
- Registrar la ingesta de alimentos con el fin de controlar y evaluar las calorías consumidas por el usuario.
- Implementar un sistema de seguimiento del progreso físico.
- Generar reportes personalizados de progreso físico.

1.4. Área de conocimiento

Área: Programación Móvil

Subárea: Sistema de Información

1.5. Alcance

- El sistema funcionará en sistemas operativos Android.
- Interfaz intuitiva para el registro de usuarios.
- Guía sobre el uso correcto de máquinas de gimnasio.
- Registrar el progreso físico (peso, estatura, edad, masa corporal).
- Creación de rutinas personalizadas según los objetivos del usuario.
- El registro de alimentación se limitará solo al conteo calórico, es decir una calculadora de IMC.
- El sistema no reemplaza la supervisión profesional de entrenadores o nutricionistas.
- No se contempla la integración con dispositivos externos (smartwatch, banda de frecuencia cardíaca, entre otros).

1.6. Justificación

1.6.1. Justificación técnica

El proyecto es técnicamente viable gracias a la combinación de tecnologías modernas y flexibles. Se empleará React Native con Expo para garantizar una experiencia fluida en dispositivos Android, permitiendo un desarrollo multiplataforma con una sola base de código.

La base de datos y autenticación estarán gestionadas por Firebase, lo que asegura escalabilidad, sincronización en tiempo real y seguridad en el manejo de datos de usuarios.

El componente de inteligencia artificial se implementará para generar rutinas personalizadas, basándose en parámetros como peso, altura, edad, nivel de experiencia y objetivos físicos. Este módulo podrá entrenarse mediante modelos preexistentes o redes neuronales ligeras optimizadas para dispositivos móviles.

Además, se integrará un sistema de visualización interactiva con imágenes o animaciones que muestran la postura correcta en cada ejercicio, lo cual aumenta la comprensión y

reduce el riesgo de lesiones.

En conjunto, estas tecnologías garantizan un sistema robusto, escalable y con potencial de expansión futura.

1.6.2. Justificación económica

La aplicación representa una alternativa de bajo costo frente a los servicios tradicionales de entrenamiento personalizado. Al automatizar la generación de rutinas mediante inteligencia artificial, se reducen los costos asociados a la contratación de entrenadores físicos, permitiendo que los usuarios obtengan orientación profesional sin incurrir en gastos adicionales.

Para los gimnasios, la herramienta puede convertirse en un complemento estratégico que optimice el tiempo de los entrenadores y mejore la retención de clientes, al ofrecerles una plataforma moderna de autogestión del entrenamiento. Asimismo, el desarrollo de esta aplicación se apoya en tecnologías de código abierto (React Native, Firebase, TensorFlow Lite, etc.), lo que disminuye considerablemente los costos de desarrollo y mantenimiento del sistema.

1.6.3. Justificación social

Este proyecto busca eliminar las barreras que limitan la práctica constante del ejercicio físico, ofreciendo orientación y motivación a través de la tecnología. Está dirigido principalmente a usuarios principiantes o personas que no tienen acceso a un entrenador personal. Contribuye a la promoción de un estilo de vida saludable, fomenta la constancia y previene lesiones al enseñar el uso correcto de las máquinas.

CAPÍTULO II: MARCO TEÓRICO

2.1. Metodología de Desarrollo

Una metodología de desarrollo de software es un marco de trabajo que se usa para estructurar, planificar y controlar el proceso de desarrollo de sistemas de información (Gabriel Maide & Pacienza, 2020). Asimismo el objetivo de las distintas metodologías es el intentar organizar los equipos de trabajo para que estos desarrollen las funciones de un programa de la mejor manera posible (Santander, 2020). Una metodología establece un conjunto de prácticas, técnicas y procesos que permiten mejorar la eficiencia del trabajo, garantizar la calidad y reducir la incertidumbre durante el ciclo de vida del software (Pressman, 2010).

Las metodologías de desarrollo proporcionan lineamientos que facilitan la coordinación del equipo, la definición clara de los requerimientos, la gestión de riesgos y la evaluación del avance del proyecto. El uso de la metodología adecuada contribuye a la minimización de riesgos, estandarización de procedimientos y mejorar la productividad.

Las metodologías de desarrollo de software son enfoques que proporcionan un marco claro para planificar, ejecutar y entregar proyectos de software, estas pueden ser lineales o iterativas, se ajustan a diferentes tipos de proyectos y equipos de acuerdo a las necesidades específicas de cada proyecto o equipo (Team, 2024).

2.2. Metodologías de desarrollo de software tradicionales vs ágiles

Las metodologías de desarrollo de software se agrupan principalmente en tradicionales y ágiles, cada una con enfoques y características distintas.

2.2.1. Metodologías tradicionales

Las metodologías tradicionales, conocidas también como predictivas, son enfoques secuenciales y lineales, basados en una planificación detallada y estructurada, donde cada fase debe completarse antes de iniciar la siguiente, tampoco se puede volver hacia atrás una vez se ha cambiado de fase. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente (Santander, 2020).

Este enfoque es ideal para proyectos en los que los requerimientos están bien definidos desde el principio.

Tabla 2-1. Comparación de Metodologías tradicionales de desarrollo de software

Metodología	Descripción	Características
Waterfall	La metodología waterfall o cascada, es un proceso secuencial, el proyecto se divide en fases, las cuales deben completarse completamente antes de pasar a la siguiente. El avance del progreso va de arriba hacia abajo.	▪ Enfoque lineal y estructurado. ▪ Requiere una planificación previa. ▪ No permite adaptaciones una vez iniciada la ejecución. ▪ Fácil de entender.
Espiral	Esta metodología integra la gestión de riesgos a lo largo del proceso de desarrollo. El desarrollo ocurre en ciclos o vueltas y cada uno de estos cuenta con 4 fases: planificación, análisis de riesgos, ingeniería y evaluación por parte del cliente.	▪ Enfoque iterativo basado en ciclos sucesivos. ▪ Permite incorporar ajustes o mejoras en cada iteración. ▪ Contempla análisis de riesgos periódico.
Incremental	Esta metodología combina elementos de la metodología cascada, permite desarrollar el producto de forma progresiva, cada etapa incremental constituye una nueva funcionalidad, lo que permite ver resultados de forma más rápida en comparación del modelo cascada.	▪ Entrega parcial del sistema. ▪ Facilita la detección de errores temprana. ▪ Recopila comentarios de clientes después de cada entrega. ▪ Flexible a los cambios. ▪ Combinación de enfoques lineales e iterativos.
Prototipado	Esta metodología desarrolla un modelo preliminar y tangible de software, esto permite que el usuario pruebe y de retroalimentación, para así poder ajustar los requisitos y mejorar el diseño del sistema final.	▪ Facilita la comunicación con el usuario. ▪ Reduce la incertidumbre. ▪ Es iterativa. ▪ Reduce riesgos y costos. ▪ Es flexible y versátil.

Metodología	Descripción	Características
Diseño rápido de aplicaciones RAD	Esta metodología permite desarrollar software de alta calidad en un corto periodo de tiempo. Los costes son mucho más altos y el desarrollo es flexible, aunque requiere mayor intervención de los usuarios.	■ Desarrollo basado en prototipos funcionales. ■ Iteraciones cortas y desarrollo incremental. ■ Alta participación del usuario.

Fuente: Elaboración propia.

Conclusión: De acuerdo al análisis comparativo presentado en la Tabla 2-1, se concluye que la principal diferencia entre las metodologías tradicionales radica en su tolerancia al cambio y la gestión de riesgo. Mientras que el modelo cascada prioriza una estructura secuencial y rígida, los modelos espiral, incremental y prototipado introducen cambios iterativos que permiten la validación progresiva del usuario. Por lo tanto, estas metodologías son más aptas para proyectos con requerimientos claros, estables que no cambiaran con el tiempo.

2.2.2. Metodologías ágiles para móvil

Las metodologías ágiles aplicadas al desarrollo móvil surgen como respuesta a la necesidad de adaptarse a un entorno cambiante y a ciclos de producción rápidos. Estas metodologías están centradas en la entrega continua de valor, la colaboración constante con el cliente y la capacidad de adaptarse a nuevos requerimientos.

Las metodologías ágiles son las más utilizadas en la actualidad debido a su alta flexibilidad y agilidad (Santander, 2020). Se basan en la metodología incremental, en la que en cada ciclo de desarrollo se agregan nuevas funcionalidades a la aplicación final.

Tabla 2-2. Comparación de metodologías ágiles móviles

Metodología	Descripción	Características
Scrum	Es una metodología ágil, que divide los requerimientos y tareas de forma similar a Kanban. Está compuesto por iteraciones fijas y cortas denominadas Sprints, así como una estructura basada en roles, eventos y artefactos definidos. Su objetivo es entregar incrementos funcionales del producto en ciclos de 2 a 4 semanas, fomentando la colaboración y la inspección continua (Schwaber & Sutherland, 2020).	■ Roles definidos (Product Owner, Scrum Master, Equipo). ■ Iteraciones cortas y fijas. ■ Entregas incrementales al final de cada sprint. ■ Alta colaboración con el cliente. ■ Uso de backlog y tableros para la gestión visual del trabajo. (Schwaber & Sutherland, 2020)
Extreme Programming (XP)	Es una metodología basada en relaciones interpersonales, la colaboración y la retroalimentación continua. Su objetivo principal es crear un entorno de trabajo basado en la comunicación constante, simplicidad del diseño y participación activa del cliente. Incorpora prácticas de ingeniería como pair programming, test-driven development, refactorización constante e integración continua (Beck & Andres, 2005).	■ Trabajo colaborativo. ■ Diseño sencillo y adaptables. ■ Desarrollo guiado por pruebas. ■ Refactorización continua. ■ Integración continua. ■ Entregas frecuentes y pequeñas iteraciones. ■ Planificación basada en historias de usuario. ■ Programación en parejas.
Mobile-D	Es una metodología ágil para el desarrollo de aplicaciones móviles para equipos pequeños, está centrada en ciclos de desarrollo rápidos y entrega continua. Combina prácticas ágiles con procesos orientados al hardware y ciclos muy cortos de entrega (Abrahamsson, y otros, 2004).	■ Diseñada para desarrollo móvil. ■ Iteraciones rápidas. ■ Adaptación a restricciones de hardware. ■ Enfocado en grupos pequeños.

Metodología	Descripción	Características
Kanban	Es una metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma se crea un flujo de trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto (Santander, 2020).	▪ Visualización del trabajo. ▪ Límite de trabajo en curso. ▪ Mejora continua. ▪ No requiere iteraciones fijas. ▪ Fácil implementación en equipos individuales. ▪ Gestión visual del trabajo.

Fuente: Elaboración propia a partir de (Schwaber & Sutherland, 2020); (Beck & Andres, 2005); (Santander, 2020); (Abrahamsson, y otros, 2004).

Conclusión: En conclusión, todas las metodologías ágiles comparten principios como la flexibilidad y la entrega continua, sin embargo, la metodología Mobile-D es la más adecuada para proyectos móviles por su rapidez, sus iteraciones cortas y su adaptación a las particularidades del hardware.

2.3. Mobile-D

La metodología Mobile-D es un enfoque ágil diseñado para el desarrollo de software en dispositivos móviles. Surgió como resultado de investigaciones realizadas por el VTT, con el objetivo de responder a los desafíos propios del entorno móvil, como ciclos de producción acelerados, restricciones de hardware, diversidad de plataformas y cambios frecuentes en los requisitos (Abrahamsson et al., 2004).

Mobile-D es adecuada para proyectos que requieren rapidez, flexibilidad y alta calidad, como aplicaciones de seguridad informática, plataformas de comercio electrónico, aplicaciones de simulación entre otros. Está pensada para equipos pequeños. Esta metodología combina prácticas de diversas metodologías ágiles, como Scrum, XP y prácticas de ingeniería de software.

2.3.1. Fases y etapas

El desarrollo mediante el enfoque de Mobile-D se compone de 5 fases como se muestra en la Figura 2-1, estas fases son: Exploración, Inicialización, Producción, Estabilización y Pruebas del sistema.

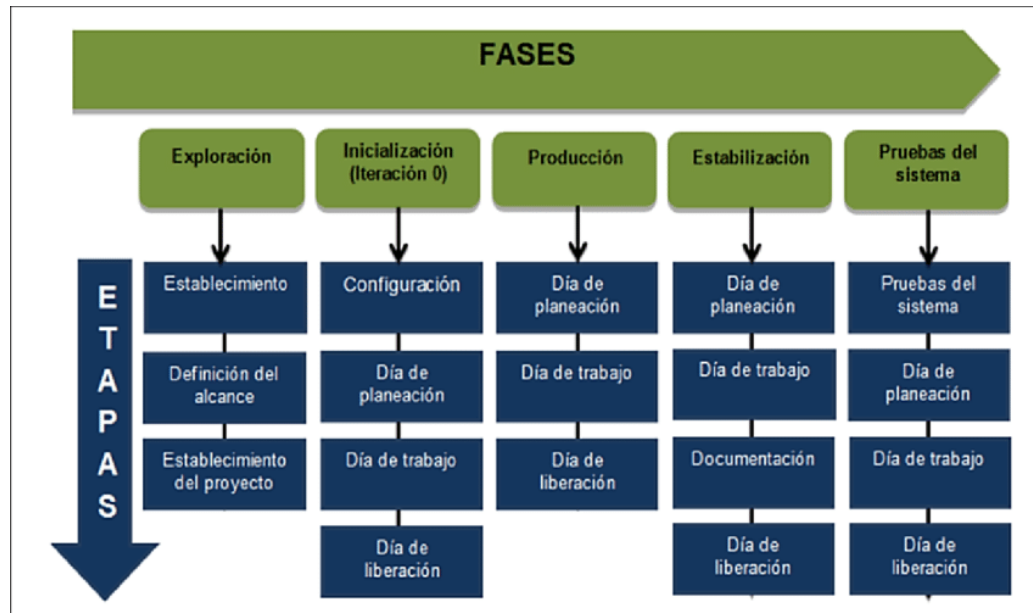


Figura 2-1. Fases y etapas de la metodología Mobile-D (Leyva, León, & García, 2016)

■ Fase de Exploración

Esta es la fase inicial donde se define el proyecto y se establecen las bases técnicas y organizativas. En esta fase se definen los objetivos, el alcance, los actores involucrados y las restricciones técnicas y funcionales del proyecto. También se seleccionan las tecnologías principales y se configura el entorno de trabajo.

■ Fase de Inicialización

En esta fase se prepara el equipo, las herramientas y la arquitectura técnica para comenzar las iteraciones de desarrollo. Dentro de esta fase se verifican los recursos humanos, tecnológicos y físicos que se necesitarán. Esta fase se divide en cuatro etapas: la puesta en marcha del proyecto, la planificación inicial, el día de prueba y día de salida (Gómez & Hernández, 2016).

■ Fase de Producción

Esta fase se caracteriza por el uso de ciclos cortos e incrementales. En esta etapa se implementan funcionalidades mediante prácticas de desarrollo guiado en pruebas TDD, la integración continua y el desarrollo iterativo. Su objetivo principal es construir versiones funcionales del producto de manera progresiva, garantizando la

calidad y retroalimentación constante durante todo el proceso.

- **Fase de Estabilización**

En esta fase se integran los componentes generados durante la producción para asegurar que el sistema funcione correctamente. Esta etapa los desarrolladores optimizan el rendimiento, mejoran la consistencia interna del software y realizan las refactorizaciones necesarias. No se añaden nuevas funcionalidades, sino que se fortalece la calidad general del producto.

- **Fase de Pruebas**

En esta fase se verifica que la aplicación cumpla con los requerimientos funcionales y de calidad definidos al inicio del proyecto. Esta etapa incluye pruebas de aceptación, pruebas de dispositivos reales y corrección de errores detectados. El propósito de esta etapa es garantizar que el sistema sea estable, usable y se encuentre listo para ser entregado al cliente o desplegado en un entorno final.

2.4. Programación Móvil

La programación móvil consiste en diseñar, implementar, probar y desplegar software que se ejecuta en dispositivos móviles. Este tipo de desarrollo de software tiene como objetivo crear aplicaciones que se ejecuten en varios dispositivos móviles, incluidos Android, iOS y otras plataformas menos populares. La plataforma móvil abarca dos enfoques principales de desarrollo nativo y multiplataforma (AppMaster, 2023).

Una aplicación móvil se define como un software diseñado para ejecutarse en dispositivos portátiles como teléfonos inteligentes y tabletas, con el fin de realizar tareas específicas para el usuario (Apps, 2011).

En el ámbito de desarrollo de software, los sistemas operativos móviles dominantes son Android y iOS, los cuales concentran prácticamente la totalidad del mercado global de smartphones. Android, mantenido por Google, es un sistema abierto basado en Linux que ofrece gran flexibilidad, soporte para múltiples fabricantes y acceso a herramientas de desarrollo como Android Studio y las bibliotecas Jetpack (Google, 2023). Por su parte, iOS desarrollado por Apple, se caracteriza por ser un sistema operativo cerrado, de alto control de calidad, diseñado exclusivamente para ejecutarse en el hardware de Apple y herramientas como Xcode y SwiftUI que permiten crear aplicaciones con elevado rendimiento y experiencia de usuario (Apple, 2023).

Tabla 2-3. Comparación de Android vs iOS

Características	Android	iOS
Propietario	Google	Apple
Familia S.O.	Linux	UNIX
Tipo de licencia	Código abierto	Código cerrado y propietario
Programado en	C, C++, Java	Swift, Objective-C, C, C++
Lenguajes de programación	Java, Kotlin	Swift
Entorno de desarrollo	Android Studio	Xcode
Arquitectura del sistema	Basada en kernel Linux + HAL + Runtime ART	Basada en kernel XNU + frameworks de alto nivel
Presencia en el mercado boliviano	90.74 % (Global, 2025)	8.7 % (Global, 2025)
Diseño	Diseños flexibles, personalizado, colorido, adaptable a muchas marcas y dispositivos.	Diseño minimalista, elegante, limpio y extremadamente consistente.
Personalización	Permite la personalización de la interfaz mediante launchers personalizados, permite utilizar paquetes de temas, ajustar el panel de configuración rápida y modificar el comportamiento de aplicaciones predeterminadas (Gironés, 2012).	Su personalización es limitada, prioriza la simplicidad, estabilidad y coherencia visual, lo que reduce riesgos de incompatibilidad (Apple, 2023).
Fragmentación	Samsung, Huawei, Xiaomi, Motorola, Sony.	Exclusivo para dispositivos Apple (iPhone, iPad).
Distribución de Apps	Play Store y permite instalación manual (APK/Sideload).	App Store
Asistente Virtual	Google Now, Google Assistant	Siri

Fuente: Elaboración propia.

Conclusión: En conclusión, Android destaca por su capacidad de adaptabilidad, y su compatibilidad con distintos lenguajes de programación, lo que lo hace más versátil; su flexibilidad y escalabilidad lo hacen la plataforma idónea para el desarrollo de una aplicación móvil.

2.5. Sistema Operativo

2.5.1. Android

Android es un sistema operativo y una plataforma de desarrollo libre basada en Linux y de código abierto desarrollado por Google (Tomás, 2019). Este sistema operativo es utilizado en tablets, netbooks, reproductores de música e incluso, PC's (Sanz, 2019). Hoy en día podemos encontrar relojes, gafas, cámaras, TV, sistema para automóviles, electrodomésticos y una gran variedad de sistemas empujados que se basan en este sistema operativo por lo que lo hace adaptable a cualquier tipo de hardware (Tomás, 2019).

Su dominio en el mercado global, superando el 70 % de cuota del mercado, y su prevalencia superior al 90 % en el contexto boliviano, lo posicionan como el estándar en el desarrollo Mobile en la región (Global, 2025). El desarrollo Android ha ido evolucionando puesto que permite que fabricantes, desarrolladores y la comunidad global adapten el sistema, añadiendo funciones nuevas y personalicen la interfaz, lo que facilita la creación de aplicaciones innovadoras y la configuración del sistema a distintos entornos y necesidades (Alliance, 2022).

2.5.2. Arquitectura de Android

Desde una perspectiva técnica, Android se compone de cuatro capas principales. En la base se encuentra el kernel de Linux, responsable de gestionar procesos, la administración de memoria y la seguridad del sistema. Sobre él se ubica la capa del Hardware Abstraction Layer (HAL), que permite que diferentes fabricantes integren su propio hardware. La tercera capa de las APIs de Android, que ofrecen servicios como gráficos, multimedia, ubicación y sensores. Finalmente, el framework de aplicaciones, que facilita la creación de aplicaciones mediante bibliotecas de alto nivel y componentes como Activities, Services y Content Providers (Meier & Lake, 2018).

El siguiente gráfico muestra la arquitectura de Android. Como se puede observar en la Figura 2-2, su arquitectura se encuentra formada por cuatro capas.

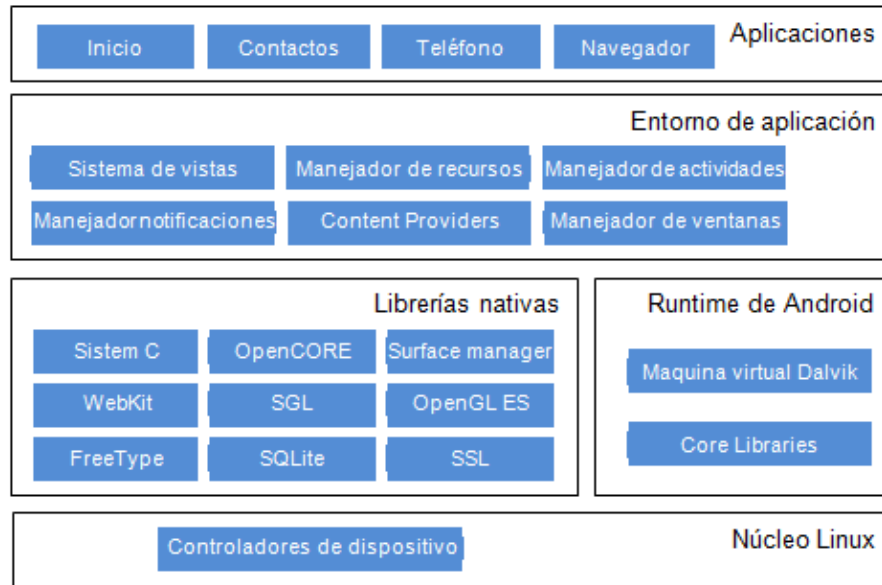


Figura 2-2. Arquitectura de Android (Gómez A. A., 2015)

2.5.3. Ventajas y limitaciones

Android destaca por que ofrece ventajas como:

- Flexibilidad para funcionar en múltiples tipos de hardware.
- Libertad de personalización de interfaz.
- Libertad de configuración del sistema.
- Compatibilidad amplia con una gran variedad de herramientas y servicios de Google.
- Alta calidad de gráficos y sonido.
- Portabilidad garantizada, ya que sus aplicaciones son desarrolladas en Java.
- Nivel de seguridad aceptable.

Sin embargo, esto también genera limitaciones, como la fragmentación entre dispositivos y diferentes velocidades en la distribución de actualizaciones del sistema.

2.6. Realidad Aumentada

La realidad aumentada (RA) es una combinación visual de elementos reales y virtuales que interaccionan entre ellos (Navarro, Martínez, & Martínez, 2018), también es la integración en tiempo real de la información digital en el entorno de un usuario. La tecnología de RA superpone el contenido al mundo real, lo que enriquece la percepción de la realidad

del usuario en lugar de reemplazarla (Hayes & Downie, 2025). A diferencia de la realidad virtual, que sustituye completamente el entorno físico, la RA superpone información digital como imágenes, modelos 3D, texto o animaciones sobre la realidad observada a través de dispositivos como smartphones, tablets o lentes inteligentes (Azuma, 1997).

2.6.1. Componentes de un sistema de Realidad Aumentada

Un sistema de Realidad Aumentada es una convergencia de hardware y software que opera un bucle continuo de retroalimentación, en otras palabras, un sistema de RA debe cumplir con tres características principales, combinar lo real y lo virtual, operar interactivamente y registrar elementos en 3D (Azuma, 1997).

Un sistema de RA incluye los siguientes componentes esenciales:

- **Dispositivo de visualización:** Permite mostrar la mezcla entre la imagen real y los elementos virtuales. Puede ser un teléfono móvil, tablet, gafas inteligentes o pantalla montada en la cabeza (Head-Mounted Display).
- **Cámara o sensores de captura:** Registra el entorno físico para detectar superficies, marcadores o puntos de referencia. Por ejemplo: cámaras RGB, sensores de profundidad, giroscopio, acelerómetro.
- **Módulo de seguimiento (tracking):** Se encarga de identificar la posición y orientación del usuario o del dispositivo. Esto incluye: marker-based, markerless, SLAM, reconocimiento de superficies.
- **Motor de renderizado:** Genera y superpone objetos virtuales en el entorno real con coherencia visual y espacial.
- **Interfaz de interacción:** Permite al usuario manipular objetos virtuales mediante gestos, toques o movimiento.

2.6.2. Tecnologías para el desarrollo de Realidad aumentada móvil

Para el desarrollo móvil, las principales tecnologías de RA son:

- **ARCore:** Es el SDK de realidad aumentada de Google que ofrece APIs multiplataforma para crear nuevas experiencias envolventes en Android, iOS, Unity y la Web. Transforma la manera en que las personas juegan, compran, aprenden, crean y experimentan el mundo juntas mediante la comprensión contextual de las personas, los lugares y las cosas (Developers, 2025).
- **ARKit:** Es un framework de Apple, altamente optimizado y compatible con sensores

avanzados como LiDAR. Permite capturar impresionantes videos de alta resolución perfectos para la edición de video profesional, producción cinematográfica, aplicaciones de redes sociales entre otros (Inc., 2025).

2.7. Inteligencia Artificial

La inteligencia artificial (IA) es una tecnología transformadora que permite a las máquinas llevar a cabo tareas de resolución de problemas similares a las de los humanos. Desde el reconocimiento de imágenes y la generación de contenido creativo hasta la creación de predicciones basadas en datos, la IA permite a las empresas tomar decisiones más inteligentes a escala (Services, 2025).

La IA consiste en enseñar a las computadoras a hacer las actividades increíbles que nuestros cerebros pueden hacer, desde comprender el mundo que las rodea hasta aprender temas nuevos y generar ideas originales (Cloud, 2025). En el contexto de las aplicaciones móviles, la IA permite integrar funciones avanzadas como la personalización del contenido, la detección automática de movimiento, el análisis de rendimiento del usuario y la interacción mediante voz a texto.

2.7.1. Aprendizaje automático

El aprendizaje automático o Machine Learning es una subárea de la inteligencia artificial dedicado a comprender y construir métodos para imitar la forma en que aprenden los humanos (Pykes, 2024). El aprendizaje automático, es la capacidad que tiene un programa para mejorar su desempeño en una tarea a partir de la experiencia acumulada (Mitchell, 1997).

2.7.2. Redes neuronales

Una red neuronal es un programa o modelo de machine learning que toma decisiones de forma similar al cerebro humano, al emplear procesos que imitan la forma en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones. Se basan en datos de entrenamiento para aprender y mejorar su precisión con el tiempo (IBM, 2025).

2.7.3. Visión por computadora

La visión artificial es un campo de la inteligencia artificial, que permite a los ordenadores y sistemas interpretar y analizar datos visuales, así como también obtener información pertinente a partir de imágenes digitales, videos y otros datos visuales (Cloud, 2025). En el contexto de las aplicaciones móviles modernas, permite interpretar información visual mediante algoritmos que detectan objetos, poses, personas entre otras características del entorno.

2.7.4. Modelos de lenguaje Extenso (LLM)

Los modelos de lenguaje, son modelos básicos entrenados sobre inmensas cantidades de datos, lo que los hace capaces de comprender y generar lenguaje natural y otros tipos de contenido para realizar una amplia variedad de tareas. Por lo general, los LLM se basan en arquitecturas de aprendizaje profundo, como Transformer, desarrollado por Google en 2017, y se pueden entrenar con miles de millones de textos y otros tipos de contenido.

Debido a su escalabilidad, los LLM se pueden entrenar con otros tipos de datos, como código, imágenes, audio y video, entre otros (Cloud, 2025). Su versatilidad y capacidad de aprendizaje los convierten en herramientas clave en el desarrollo de aplicaciones modernas, incluyendo sistemas móviles inteligentes y soluciones que integren realidad aumentada e inteligencia artificial.

2.8. Fundamentos del entrenamiento físico

2.8.1. Entrenamiento físico

El entrenamiento es un conjunto de ejercicios y actividades que se realizan regularmente para mejorar la salud y el estado físico de una persona. Este tipo de entrenamiento puede incluir variedad de actividades, cada una enfocada en diferentes aspectos del cuerpo y objetivos personales (Orientanet, s.f.).

Hoy en día, la tecnología como sensores, aplicaciones móviles, realidad aumentada e inteligencia artificial permiten un monitoreo exhaustivo de la técnica y el análisis del movimiento, ofreciendo retroalimentación personalizada en tiempo real. Estas innovaciones no solo optimizan el rendimiento de los usuarios, sino que también garantizan que quienes están comenzando su camino en el ejercicio puedan realizar las actividades de manera segura y eficaz, contribuyendo de esta manera a la reducción de riesgo de lesiones y fomentando un

estilo de vida saludable.

2.8.2. Principios del entrenamiento físico

El entrenamiento físico está fundamentado en tres principios esenciales, estos son:

- **Fuerza:** Se centra en incrementar la capacidad del músculo para generar tensión, mediante ejercicios con resistencias progresivas. Esto no solo mejora el rendimiento deportivo, sino que también contribuye a la salud articular y la densidad ósea.
- **Técnica:** Implica la ejecución correcta de cada movimiento para maximizar los beneficios y minimizar riesgos. Una técnica adecuada asegura que el estímulo recaer sobre los músculos deseados y evita compensaciones que puedan generar lesiones.
- **Hipertrofia:** Busca aumentar el tamaño del músculo mediante cargas, repeticiones y volúmenes adecuados. La hipertrofia requiere un equilibrio entre sobrecarga progresiva, nutrición y recuperación para generar crecimiento muscular efectivo.

2.8.3. Biomecánica básica

La biomecánica estudia el movimiento del cuerpo humano aplicando principios mecánicos. Comprenderla permite optimizar los ejercicios, analizar palancas, ángulos de articulaciones y fuerza aplicada, lo que mejora la eficiencia y reduce el riesgo de lesiones. Los aspectos clave incluyen la alineación corporal, el centro de gravedad, estabilidad y amplitud de movimiento.

La biomecánica proporciona las bases para corregir patrones de movimiento disfuncionales, los cuales pueden surgir por hábitos sedentarios, desequilibrios musculares o mala ejecución repetida de ejercicios. Identificar estos patrones permite intervenir de manera temprana mediante correcciones técnicas, fortalecimiento de músculos estabilizadores o ajustes en la postura.

2.8.4. Lesiones comunes por mala técnica

La técnica incorrecta en la ejecución de ejercicios, es una de las principales causas de las lesiones en entornos de entrenamiento físico, especialmente en principiantes. La evidencia científica demuestra que una postura inadecuada, la falta de control motor y el uso de cargas excesivas incrementan el riesgo de daño musculoesquelético (Kolt & Snyder-Mackler, 2007).

El desconocimiento de la técnica o la negligencia en la postura pueden provocar lesiones

frecuentes, como:

- Dolor lumbar por levantamiento incorrecto de peso.
- Tendinitis por movimientos repetitivos sin control.
- Esguinces o distensiones musculares por falta de calentamiento o postura incorrecta.
- Lesiones articulares en hombros, rodillas o muñecas por sobrecarga o ángulos inadecuados.

Prevenir estas lesiones requiere educación, supervisión y el uso de tecnologías de monitoreo como sensores de movimiento y análisis de video.

2.8.5. Máquinas de gimnasio

Una máquina de gimnasio es un equipo diseñado para facilitar la realización de ejercicios físicos de manera controlada, segura y específica. Estas máquinas están construidas con estructuras metálicas, poleas, cables, placas de peso o mecanismos electrónicos que permiten guiar el movimiento del usuario. Su diseño busca aislar determinados grupos musculares, mejorar la estabilidad durante el ejercicio y proporcionar una carga regulable para adaptarse al nivel de cada persona.

Las máquinas de gimnasio sirven para entrenar diferentes capacidades físicas como fuerza, resistencia muscular, estabilidad y acondicionamiento cardiovascular. Proporcionan un entorno controlado donde el usuario puede ejecutar movimientos guiados que reducen el riesgo de lesiones, especialmente en personas principiantes o en rehabilitación. También permiten trabajar grupos musculares específicos con mayor precisión, mejorar la postura durante el ejercicio y mantener una progresión segura mediante cargas ajustables. Además, algunas máquinas monitorean parámetros como ritmo cardíaco, velocidad o calorías, facilitando el seguimiento del rendimiento.

Las máquinas de gimnasio pueden clasificarse de diversas formas, pero las categorías más comunes son:

■ Según su función principal

- *Máquinas de fuerza o resistencia:* utilizan poleas, cables, placas de peso o sistemas hidráulicos para trabajar grupos musculares específicos. Ejemplos: prensa de piernas, máquina de pecho, jalón al pecho, curl de bíceps.
- *Máquinas cardiovasculares:* diseñadas para mejorar la resistencia cardiorrespiratoria y quemar calorías. Ejemplos: caminadoras, bicicletas estáticas, elípticas,

remadoras.

■ **Según el tipo de movimiento**

- *Movimiento guiado*: la máquina controla la trayectoria del movimiento. Son ideales para principiantes.
- *Movimiento libre asistido*: permiten mayor libertad, pero brindan estabilidad adicional.

■ **Según el mecanismo de resistencia**

- Placas de peso (weight stack): se selecciona la carga con un pasador.
- Peso libre integrado: discos que se colocan manualmente.
- Sistema hidráulico: resistencia generada por pistones de aire o aceite.
- Electromagnéticas: comunes en máquinas cardiovasculares modernas.

■ **Según el área del cuerpo que trabajan**

- Máquinas para tren superior: pecho, espalda, hombros, brazos.
- Máquinas para tren inferior: piernas, glúteos.
- Máquinas para core: abdominales, zona lumbar.

2.9. Plataforma de desarrollo

2.9.1. Visual Studio Code



Visual Studio Code es un editor de código fuente creado por Microsoft, diseñado para ser ligero, rápido y altamente personalizable (Cuadrado, 2022). Sirve para escribir, corregir y organizar programas en muchos lenguajes como Java, Python, C++, entre otros. Este funciona en Windows, Mac y Linux.

A diferencia de un IDE tradicional que viene completo desde la instalación, VS Code parte de un núcleo ligero y va incorporando funcionalidad mediante extensiones, lo que permite adaptarlo a prácticamente cualquier lenguaje o framework sin cargar con herramientas que el proyecto no necesita.

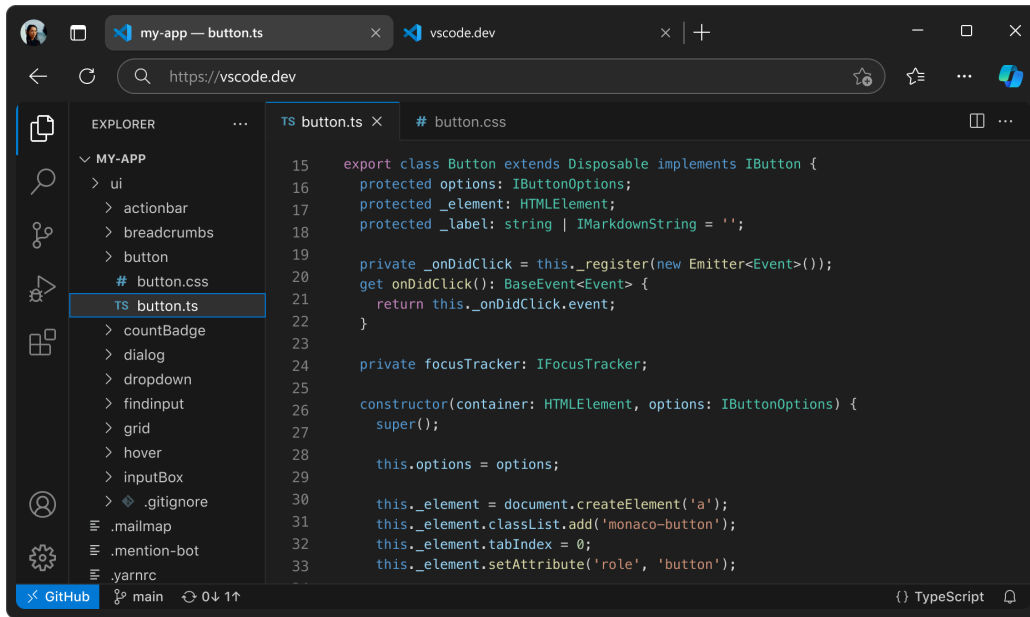


Figura 2-3. Interfaz de Visual Studio Code

2.10. Frameworks para desarrollo Mobile

El desarrollo de aplicaciones móviles ha evolucionado en gran manera gracias a la aparición de frameworks que permiten crear interfaces eficientes, multiplataforma y optimizadas para dispositivos con recursos limitados. Estos frameworks ofrecen herramientas, bibliotecas y entornos de trabajo que simplifican el proceso de creación de aplicaciones al abstraer componentes nativos y proporcionar estructuras reutilizables.

Entre los frameworks más utilizados se encuentran React Native, Flutter, Ionic, Xamarin y Jetpack Compose.

Tabla 2-4. Comparación de frameworks de desarrollo Mobile

Framework	Descripción	Características	Ventajas
React Native	Es un framework de Meta para construir aplicaciones nativas para iOS y Android, usando JavaScript. Este framework usa el paradigma fundamental de construcción de bloques de UI (componentes visuales con los que interacciona el usuario) (Deloitte, 2019).	▪ Multiplataforma. ▪ Fast Refresh y Hot Reload. ▪ Acceso a componentes y APIs nativas. ▪ Gran comunidad y ecosistema amplio. ▪ Integración con librerías como Redux, Axios, Firebase.	Alto rendimiento. Reutilización de código. Amplia compatibilidad.
Flutter	Es un framework de código abierto de Google basado en Dart con motor de render propio (Flutter, s.f.).	▪ Widgets personalizables. ▪ Animaciones. ▪ Hot Reload.	UI consistente. Excelente rendimiento gráfico.
Ionic	Ionic es un framework híbrido basado en tecnologías web (HTML, CSS, JavaScript) que funciona mediante WebView. Permite construir aplicaciones con enfoques multiplataforma usando Angular, React o Vue. Es especialmente útil para aplicaciones orientadas a contenidos web o prototipado rápido (Ionic Documentation, 2023).	▪ Multi-framework. ▪ Amplia colección de componentes UI. ▪ Soporte para Progressive Web Apps.	Desarrollo rápido. Curva de aprendizaje baja.

Framework	Descripción	Características	Ventajas
Xamarin	Xamarin es un framework de Microsoft basado en C# y .NET, que compila aplicaciones nativas para Android, iOS y Windows. Permite compartir hasta 90 % de la lógica de negocio entre plataformas y se integra profundamente con Visual Studio. Es utilizado ampliamente en entornos empresariales (Microsoft, 2022).	▪ Acceso directo a APIs nativas. ▪ Integración con Visual Studio. ▪ Integración con .NET MAUI.	Sólido para apps empresariales. Reutilización de lógica.
Jetpack Compose	Jetpack Compose es el framework declarativo moderno de Android, basado en Kotlin. Permite construir interfaces de usuario mediante funciones composables, eliminando XML y ofreciendo mayor control del sistema y rendimiento nativo. Es el estándar actual para desarrollar aplicaciones Android modernas (Android Developers, 2024).	▪ UI declarativa. ▪ Integración con Android Jetpack. ▪ Animaciones avanzadas.	Rendimiento máximo. Control total del sistema.

Fuente: Elaboración propia basada en (Deloitte, 2019); (Flutter, s.f.); (Android Developers, 2024); (Microsoft, 2022).

Conclusión: En conclusión, React Native destaca frente a los demás frameworks debido a su equilibrio entre rendimiento y desarrollo multiplataforma, su capacidad de generar componentes nativos le da mayor fluidez a diferencia de Ionic y Flutter. Así también, permite desarrollar aplicaciones de forma eficiente y se adapta al entorno de Android.

2.10.1. React Native

React Native es un framework para la creación de aplicaciones nativas para iOS y Android, creado por Meta (Facebook). Permite construir aplicaciones multiplataforma utilizando

JavaScript y React. A diferencia de las soluciones híbridas tradicionales, React Native genera interfaces nativas, lo que permite lograr un rendimiento cercano al desarrollo completamente nativo (React Native, 2025).

React Native se caracteriza por (Viera, s.f.):

- **Componentes nativos:** Usa componentes de interfaz de usuario nativo en lugar de componentes web.
- **Hot reloading:** Permite ver cambios en tiempo real sin recompilar toda la app, acelerando el desarrollo.
- **Acceso a APIs nativas:** Permite integrar funcionalidades como cámara, GPS y notificaciones directamente del dispositivo.
- **Desarrollo Multiplataforma:** Escribe una sola vez y ejecuta en ambas plataformas (iOS y Android).
- **Gran Comunidad:** Amplia comunidad de desarrolladores y recursos disponibles.
- **Rendimiento:** Ofrece un rendimiento cercano al nativo gracias a su arquitectura.
- **Interfaz de Usuario:** Permite crear interfaces de usuario ricas y fluidas.
- **Integración con Librerías:** Compatible con librerías de terceros y APIs nativas.
- **Soporte para TypeScript:** Permite usar TypeScript para un desarrollo más robusto y seguro.

2.11. Bases de datos de aplicaciones Móviles

Las bases de datos desempeñan un papel fundamental en el desarrollo de aplicaciones móviles, ya que permiten almacenar, administrar y sincronizar la información que utiliza el sistema durante su funcionamiento.

En el entorno móvil actual existen múltiples opciones tecnológicas, entre las cuales destacan Firebase Firestore, SQLite, MongoDB Atlas, MySQL y PostgreSQL.

Tabla 2-5. Comparación de bases de datos para aplicaciones móviles

Base de Datos	Descripción	Características	Ventajas
Firebase Firestore	Es una base de datos NoSQL en la nube que permite sincronización en tiempo real y almacenamiento distribuido (Isaac, 2025). Su base de datos principal, Cloud Firestore, es una solución NoSQL escalable diseñada para sincronización en tiempo real, rendimiento móvil y arquitectura serverless.	▪ Escalable. ▪ Documentos y colecciones. ▪ Soporte offline.	▪ Fácil integración móvil. ▪ Sin servidor. ▪ Arquitectura serverless. ▪ Integración con el ecosistema de Google.
SQLite	Es una biblioteca en lenguaje C, es el motor de base de datos más utilizado en el mundo. Se encuentra integrado en todos los teléfonos móviles y en la mayoría de las computadoras, es de código abierto (SQLite Consortium, 2024).	▪ Escalable. ▪ Documentos y colecciones. ▪ Soporte offline.	▪ Fácil integración móvil. ▪ Ligera y de bajo consumo. ▪ Amplio soporte y estabilidad. ▪ Sin servidor. ▪ Tiempo real.
MongoDB Atlas	Es un servicio de base de datos multicloud, simplifica la implementación y gestión de bases de datos. Permite manejar grandes volúmenes de datos, consultas avanzadas y procesamiento en la nube con altos niveles de seguridad y escalabilidad (MongoDB, 2024) (Huang, 2020).	▪ Ligero. ▪ Transaccional. ▪ Estructura relacional.	▪ Ideal para uso offline. ▪ Ampliamente soportado. ▪ Integración nativa con la nube. ▪ Funciones avanzadas de seguridad.

Base de Datos	Descripción	Características	Ventajas
MySQL	MySQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto ampliamente utilizado en aplicaciones web y móviles. Originalmente desarrollado por MySQL AB y actualmente mantenido por Oracle Corporation, MySQL se ha consolidado como una solución flexible, rápida y confiable para gestionar datos estructurados (DuBois, 2013); (Corporation, 2024).	▪ Lenguaje SQL para definición y manipulación de datos. ▪ Soporte para múltiples motores de almacenamiento. ▪ Ejecución de transacciones ACID. ▪ Alta compatibilidad con lenguajes de programación.	▪ Flexible. ▪ Escalable. ▪ Adecuada para grandes volúmenes.
PostgreSQL	Sistema de gestión de bases relacional ampliamente utilizado. Destaca por su arquitectura orientada a objetos, su capacidad para manejar grandes volúmenes de datos y su soporte para consultas complejas, extensiones y tipos de datos personalizados.	▪ SQL. ▪ Robusto. ▪ Seguro. ▪ Gestión avanzada de concurrencia. ▪ Soporte para JSON y JSONB para datos semi-estructurados.	▪ Ideal para Backend estructurado. ▪ Rendimiento estable. ▪ Perfecto para aplicaciones móviles.

Fuente: Elaboración basada en (Corporation, 2024); (Huang, 2020); (MongoDB, 2024).

Conclusión: En conclusión, Firebase Firestore es la mejor alternativa, por su arquitectura NoSQL, reduce la complejidad del Backend y es escalable.

2.11.1. Firebase

Firebase es una plataforma en la nube desarrollada por Google, facilita el desarrollo de aplicaciones, bases de datos en tiempo real, autenticación y hosting. Esta herramienta

permite enviar notificaciones push y analizar el rendimiento de las apps (Isaac, 2025).

En la actualidad, Firebase integra servicios modernos como Cloud Firestore, Google Analytics 4, App Check, Data Connect y Firebase Studio, lo que la convierte en una solución completa para proyectos actuales que buscan escalabilidad, seguridad y analítica avanzada (López Mora, 2025).

2.12. Modelos de Lenguaje Extenso (LLM)

Tabla 2-6. Comparación de modelos de lenguaje extenso

Modelo	Descripción	Características	Ventajas
OpenAI GPT-4	Modelo avanzado de OpenAI con capacidades de razonamiento y comprensión contextual profunda.	Alta precisión; multimodal; entrenado con gran cantidad de parámetros.	Respuestas coherentes; excelente para tareas complejas.
Gemini (Google)	Modelo multimodal de Google capaz de procesar texto, imágenes y audio.	Optimización contextual; integración nativa con Google.	Muy rápido; fuerte rendimiento en análisis multimodal.
Claude 3	Modelo de Anthropic centrado en seguridad y precisión.	Entrenamiento ético; respuestas estables y seguras.	Ideal para tareas largas y análisis profundo.
Llama 3	Modelo open-source de Meta, altamente eficiente y adaptable.	Open source; optimización local; adaptable.	Perfecto para personalización y despliegue en servidores propios.

Fuente: **[PENDIENTE: indicar la fuente de la tabla.]**

2.12.1. OpenAI

OpenAI ofrece modelos de lenguaje avanzados, como GPT-4, que permiten realizar tareas complejas de procesamiento de lenguaje natural, generación de texto, análisis semántico, recomendaciones personalizadas y soporte conversacional. Estos modelos se integran

mediante API y pueden ser empleados dentro de aplicaciones móviles para funciones inteligentes.

Los modelos de OpenAI están basados en la arquitectura Transformer, que permite procesar grandes volúmenes de información con alta precisión. Además, OpenAI proporciona herramientas que facilitan la integración de estos modelos en distintos entornos tecnológicos, incluyendo plataformas web, sistemas de atención al cliente, análisis de datos y automatización de procesos empresariales. Los modelos aprenden a partir de enormes conjuntos de datos, lo que les permite comprender contextos complejos, mantener conversaciones coherentes y generar contenido creativo en múltiples idiomas. Por otra parte, OpenAI trabaja en la mejora continua de la seguridad y la ética en el uso de IA, implementando mecanismos para minimizar sesgos, evitar desinformación y garantizar que los resultados sean confiables y útiles para los usuarios finales.

2.13. Herramientas

2.13.1. Control de versiones

El control de versiones es un sistema que ayuda a rastrear y gestionar los cambios realizados en un archivo o conjunto de archivos, principalmente utilizado por ingenieros de software para hacer seguimiento a las modificaciones realizadas en el código fuente (B, 2025).

Para esto utilizaremos:

1. **Git:** Es un sistema de control de versiones distribuido, gratuito y de código abierto diseñado para manejar proyectos pequeños, como también proyectos muy grandes de forma rápida y eficiente (Git, s.f.).
2. **GitHub:** Es un servicio basado en la nube que aloja un sistema de control de versiones (VCS) llamado Git. Éste permite a los desarrolladores colaborar y realizar cambios en proyectos compartidos, a la vez que mantienen un seguimiento detallado de su progreso (B, 2025).

2.13.2. Herramientas de Gestión

Las herramientas de gestión de proyectos están especialmente diseñadas para asistir a un empleado o equipo en la gestión eficaz de sus proyectos y tareas (Gurnov & Wrike (W.), 2025). Estas herramientas son utilizadas para la organización, planificación, seguimiento y aseguramiento de calidad del proyecto.

- **Trello:** Es una herramienta flexible para la gestión del trabajo, basado en tableros Kanban que facilita la organización visual de tareas mediante listas y tarjetas (Atlassian, 2023). Su enfoque intuitivo permite coordinar actividades, asignar responsabilidades y monitorear el avance de cada fase del proyecto.

CAPÍTULO III: ÁREA DE APLICACIÓN

El presente capítulo describe el enfoque metodológico adoptado para el desarrollo del sistema inteligente de entrenamiento en gimnasios con realidad aumentada e inteligencia artificial. El propósito es garantizar un proceso de construcción ordenado, iterativo y orientado a la calidad, acorde a las necesidades específicas del desarrollo móvil. Para ello, se selecciona la metodología ágil Mobile-D, diseñada especialmente para proyectos móviles con ciclos de producción rápidos, entregas frecuentes y constante retroalimentación.

3.1. Antecedentes

3.1.1. Enfoque metodológico

La metodología es el conjunto de procedimientos, técnicas y estrategias que orienta la ejecución de un proyecto de investigación o desarrollo tecnológico. En el presente trabajo, el enfoque metodológico define la manera en que se estructura el proceso de diseñar, desarrollar y evaluar el sistema. El proyecto adopta un enfoque cuantitativo-aplicado porque se medirá métricas objetivas del sistema inteligente, como también se realizará la creación de una solución tecnológica concreta mediante el uso de técnicas de ingeniería de software, inteligencia artificial y realidad aumentada. En este caso, se emplean fundamentos de biomecánica, visión por computadora, aprendizaje automático y diseño de software para desarrollar una herramienta que facilite el uso correcto de máquinas, la corrección técnica de ejercicios y la personalización del entrenamiento.

3.2. Metodología de desarrollo

La metodología Mobile-D fue seleccionada para guiar el proceso de desarrollo del sistema debido a que está específicamente diseñada para proyectos de software orientados a dispositivos móviles, caracterizados por ciclos de producción rápidos, restricciones de hardware y cambios frecuentes en los requerimientos.

Mobile-D es una metodología ágil diseñada para el desarrollo de aplicaciones móviles, que requiere ciclos de entrega cortos, validación continua, pruebas en dispositivos reales y flexibilidad ante cambios de interfaz o funcionalidad. El presente proyecto integra módulos de guía interactiva mediante imágenes, animaciones o realidad aumentada, generación de rutinas personalizadas mediante inteligencia artificial, herramientas de seguimiento físico

y una interfaz altamente intuitiva.

Mobile-D permite abordar de forma progresiva y controlada, los elementos principales del proyecto:

- La guía visual sobre el uso correcto de máquinas y la ejecución de ejercicios, que exige pruebas de usabilidad desde las primeras etapas.
- La generación de rutinas personalizadas mediante algoritmos de IA, cuya precisión y utilidad deben ajustarse iterativamente.
- Los módulos de planificación y seguimiento, que requieren refinamientos sucesivos para asegurar claridad, funcionalidad y coherencia en la experiencia del usuario.
- La interfaz móvil, que debe ser evaluada continuamente para garantizar facilidad de uso, accesibilidad y entendimiento para usuarios principiantes.

3.3. Aplicación de la Metodología Mobile-D

La metodología Mobile-D se implementa en el presente proyecto mediante cinco fases que permiten estructurar el desarrollo de la aplicación de manera incremental, verificable y orientada a la calidad. A continuación, se detalla la aplicación específica de cada fase al sistema inteligente de entrenamiento.

3.3.1. Fase de Exploración

En esta etapa se definen las bases conceptuales, técnicas y organizativas del presente proyecto.

Módulos principales del sistema

El sistema propuesto está compuesto por los siguientes módulos esenciales:

- Registro y autenticación de usuarios.
- Guía detallada del uso correcto de máquinas de gimnasio.
- Ejecución técnica correcta de ejercicios específicos.
- Rutinas personalizadas mediante inteligencia artificial.
- Registro nutricional básico (calorías diarias).
- Seguimiento del progreso físico (peso, IMC, medidas).
- Reportes estadísticos y visualización del avance.

Requerimientos de sistema

Requerimientos Funcionales

- RF1: Registrar y autenticar usuarios mediante correo y contraseña.
- RF2: Mostrar información técnica y visual de cada máquina de gimnasio.
- RF3: Generar rutinas personalizadas mediante IA según edad, peso, nivel y objetivo.
- RF4: Registrar peso, estatura, IMC y progreso físico.
- RF5: Registrar ingesta calórica diaria del usuario.
- RF6: Visualizar reportes y estadísticas del progreso físico.

Requerimientos No Funcionales

- RNF1: La aplicación debe ejecutarse en Android versión 8.0 o superior.
- RNF2: La interfaz debe ser intuitiva, rápida y adaptable a diferentes pantallas.
- RNF3: La sincronización de datos debe realizarse en tiempo real mediante Firebase.

Selección de Tecnologías

Frontend

- **React Native:** Es un framework multiplataforma, que permite desarrollar aplicaciones móviles para dispositivos Android.
- **JavaScript:** Lenguaje de programación principal para el desarrollo de las interfaces y la lógica del lado del cliente.
- **Figma:** Herramienta de diseño utilizada para crear prototipos interactivos, pantallas iniciales, facilitando la validación visual del producto.

Backend

- **Firebase Authentication:** Servicio que permite gestionar el registro, inicio de sesión y recuperación de credenciales de forma segura.
- **Firebase Firestore:** Base de datos NoSQL en la nube que proporciona sincronización en tiempo real y escalabilidad para manejar información de usuarios, ejercicios, rutinas y progreso.
- **Firebase Storage:** Servicio destinado al almacenamiento de imágenes, animaciones y videos que acompañan la guía técnica del sistema.

- **TensorFlow Lite:** Biblioteca optimizada para ejecutar modelos de inteligencia artificial directamente en el dispositivo móvil, utilizada para generar rutinas personalizadas basadas en parámetros del usuario.

3.3.2. Fase de Inicialización

En esta fase se prepara el entorno de desarrollo, la planificación inicial y la definición de la arquitectura del sistema. También se establecen los lineamientos de codificación y los mecanismos de control de versiones (Git).

Configuración del entorno

Las principales actividades a realizar para la preparación del ambiente, son:

- Instalación de Node.js.
- Instalación de Expo CLI.
- Creación de proyecto en Firebase y configuración de los servicios de autenticación, Firestore y Storage.
- Integración de la librería de TensorFlow Lite.
- Creación del repositorio en GitHub y definición de las ramas principales.

Arquitectura del sistema

La arquitectura definida para el desarrollo de la aplicación es el de cliente-servidor, donde la aplicación móvil actúa como cliente y los servicios en la nube proporcionan autenticación, almacenamiento de datos, persistencia multimedia y ejecución de algoritmos de IA.

La arquitectura cliente servidor permite garantizar escalabilidad, sincronización en tiempo real y disponibilidad permanente del servicio.

Estructura de la arquitectura

La estructura de la arquitectura del sistema se encuentra basada en capas:

Capa de Presentación Tecnologías: React Native + Expo, JavaScript, componentes UI.

Responsabilidades:

- Generar la interfaz visual de la aplicación.

- Gestionar la interacción del usuario.
- Capturar entradas como datos personales, selección de máquinas, rutinas y registro nutricional.
- Comunicar solicitudes al backend y mostrar respuestas.

Componentes clave:

- Módulo de registro y autenticación.
- Módulo de guía de máquinas y ejercicios.
- Módulo de rutinas personalizadas.
- Módulo de nutrición.
- Módulo de progreso y reportes.

Capa de Lógica de Negocio Esta se encuentra dentro de la app y está conectada a servicios externos.

Responsabilidades:

- Manejar reglas del sistema (cálculo de IMC, validaciones).
- Procesar datos antes de enviarlos a Firebase.
- Integrar y ejecutar modelos TensorFlow Lite para personalizar rutinas.
- Gestionar filtros, búsqueda de máquinas y procesamiento de ejercicios.

Componentes de negocio:

- Controlador de rutinas (IA).
- Controlador de ejercicios y máquinas.
- Controlador de progreso físico.
- Controlador nutricional.

Capa de Servicios Esta capa proporciona servicios esenciales para la operación del sistema:

Firebase Authentication:

- Registro de usuarios.

- Inicio de sesión.
- Recuperación de contraseñas.
- Seguridad y manejo de tokens.

Firebase Firestore:

- Base de datos NoSQL en tiempo real.
- Almacenamiento de rutinas, registros, progreso y máquinas.

Firebase Storage:

- Almacenamiento de imágenes, videos o animaciones educativas.
- Contenido para la guía técnica de máquinas.

Firebase Cloud Functions (opcional):

- Validación de datos.
- Preprocesamiento de peticiones.
- Actualizaciones automáticas de rutinas.

Estas funciones se integran a través de peticiones HTTP o SDK en tiempo real.

Capa de Inteligencia Artificial Tecnología: TensorFlow Lite.

Responsabilidades:

- Ejecutar modelos de recomendación ligera para rutinas.
- Ajustar parámetros según peso, nivel, objetivo y progreso del usuario.
- Proveer resultados inmediatos sin depender del servidor.

Este esquema reduce latencia, permite personalización offline y mejora la experiencia del usuario.

3.3.3. Fase de Producción

Esta es la etapa central del desarrollo. Dentro de esta etapa se implementan los módulos de IA, seguimiento físico y rutinas personalizadas. Esta fase de producción se organiza en iteraciones cortas, cada una de las cuales agrega un conjunto de funcionalidades al sistema.

Día de prueba

En este día se verifica que toda la plataforma técnica funcione correctamente antes de empezar a desarrollar funcionalidades reales.

Se debe realizar las siguientes acciones:

- **Verificación de la integración con Firebase**

Se realizarán pruebas básicas con los servicios principales de Firebase para garantizar su correcto funcionamiento:

- **Authentication:** creación de usuarios de prueba, validación de inicio de sesión y manejo de errores.
- **Firestore:** creación de colecciones temporales, escritura y lectura de documentos, y verificación de permisos básicos.
- **Storage:** carga y visualización de archivos multimedia para garantizar el acceso correcto al módulo de almacenamiento.

Estas pruebas aseguran que la aplicación pueda gestionar información real durante la fase de producción.

Validación del entorno móvil

Se verifica la correcta compilación del proyecto tanto en un dispositivo físico como en un emulador. Además, se comprueba la navegación entre pantallas iniciales y la ejecución estable del entorno Expo.

Prueba preliminar del modelo TensorFlow Lite

Se verifica la carga del modelo de inteligencia artificial y su ejecución básica en el dispositivo, con el fin de confirmar la compatibilidad entre TensorFlow Lite y la plataforma móvil utilizada.

Validación del control de versiones

Se realiza una prueba de uso del repositorio GitHub, asegurando que se pueden subir y descargar cambios correctamente, lo cual es indispensable para mantener un desarrollo organizado.

Día de salida

En este día se cierra la fase de inicialización, aquí no se realizan pruebas sino ajustes finales y organización para dejar el proyecto listo para comenzar la fase de producción.

Para esto, se debe realizar:

Organización y limpieza del proyecto

Se reorganiza la estructura interna del proyecto siguiendo la arquitectura definida, agrupando componentes, servicios, pantallas y modelos de datos en carpetas específicas. Asimismo, se eliminaron archivos temporales o de prueba utilizados en fases previas.

Configuración de variables y llaves de entorno

Se definieron archivos seguros para almacenar las credenciales de Firebase y otras configuraciones sensibles, evitando que estas sean expuestas en el repositorio de código.

Creación de ramas base de desarrollo en GitHub

Se estableció una estructura de ramas para gestionar el desarrollo:

- **main**: versión estable.
- **development**: versión en progreso.
- **feature/nombre-del-módulo**: para cada nueva funcionalidad.

Esta estructura permite un control ordenado de versiones.

3.3.4. Fase de Estabilización

Dentro de esta fase se integran y verifican los componentes desarrollados, asegurando consistencia, rendimiento y compatibilidad entre módulos. Las pruebas de usabilidad y validación funcional se intensifican.

Las actividades a realizarse en esta fase son:

- Refactorizar código para mejorar legibilidad.
- Optimizar carga de imágenes y videos.
- Ajustar tiempos de respuesta del sistema.
- Revisar el diseño visual para mejorar usabilidad.
- Corregir detalles detectados durante las pruebas en dispositivos reales.

3.3.5. Fase de Pruebas del sistema

Se ejecutan pruebas de aceptación, evaluación del modelo de IA, pruebas en dispositivos reales y corrección final de errores. El objetivo es garantizar que el sistema sea estable y

esté listo para su uso práctico.

Esto mediante las siguientes pruebas:

- **Pruebas Funcionales:** Las pruebas funcionales son un tipo de verificación que evalúa si el sistema cumple correctamente con los requerimientos establecidos, las pruebas funcionales se basan en técnicas de caja negra, donde el evaluador verifica funciones, escenarios y respuestas del sistema desde la perspectiva del usuario final (IEEE, 1990).
- **Pruebas de Usabilidad:** Las pruebas de usabilidad se enfocan en analizar qué tan fácil, eficiente y satisfactoria es la interacción de un usuario con la interfaz del sistema. Se fundamenta en principios como facilidad de aprendizaje, eficiencia, reducción de errores y satisfacción del usuario (Nielsen, 1993).
- **Pruebas de Aceptación:** Las pruebas de aceptación validan que el sistema completo cumpla con los requerimientos acordados, estas pruebas representan la etapa final de validación del producto e incluyen la ejecución de escenarios reales que reflejan el uso cotidiano de la aplicación (Pressman, 2010).

3.4. Roles

Dentro de la metodología Mobile-D, los roles cumplen la función de organizar las responsabilidades durante el desarrollo del proyecto, aun cuando éste sea desarrollado por una sola persona.

A continuación, se describen los roles adoptados en el proyecto:

Tabla 3-1. Roles establecidos por Mobile-D

Rol	Responsabilidades	Responsable
Product Owner	Define alcance, objetivos y prioridades del sistema	Erika Chino
Arquitecto Mobile	Diseña la arquitectura y selecciona las tecnologías	Erika Chino
Desarrollador móvil	Implementa módulos, interfaces y lógica de negocio	Erika Chino
Diseñador UI/UX	Diseña prototipos, navegación y experiencia de usuario	Erika Chino
QA Tester	Realiza pruebas funcionales, de usabilidad y rendimiento	Erika Chino

Fuente: Elaboración propia.

3.5. Técnicas e instrumentos de recolección de información

Para fundamentar el diseño y funcionamiento del sistema se utilizarán las siguientes técnicas:

- **Revisión documental:** Investigación de aplicaciones similares, biomecánica, técnica correcta de ejercicios y libros especializados.
- **Observación directa:** Registro del uso real de máquinas en gimnasios locales.
- **Entrevistas informales:** Conversaciones con usuarios principiantes para conocer sus dificultades reales en el gimnasio.
- **Prototipado:** Elaboración de pantallas en Figma para validar la experiencia de usuario antes del desarrollo.

CAPÍTULO IV: DESARROLLO DEL PROYECTO

4.1. Fase de Exploración

[**PENDIENTE:** Desarrollar el contenido del Capítulo IV.]

BIBLIOGRAFÍA

- Abrahamsson, P., Hanhineva, A., Hulkko, H., Ihme, T., Jäälinoja, J., Koskela, J., ... Salo, O. (2004). *Mobile-D: An Agile Approach for Mobile Application Development*. Helsinki: VTT Publications.
- Alliance, O. H. (2022). Obtenido de Android Open Source Project Overview.: <https://source.android.com/?hl=es-419>
- Android Developers, G. (2024). *Introducción a Android Studio*. Obtenido de Android Developers: <https://developer.android.com/studio/intro?hl=es-419>
- Apple. (2023). *Apple Developer Documentation*. Obtenido de <https://developer.apple.com/>
- AppMaster. (4 de septiembre de 2023). *Programación Móvil*. Obtenido de <https://appmaster.io/es/glossary/programacion-movil>
- Apps, L. B. (2011). *Guía de aplicaciones móviles*. Obtenido de Guía de aplicaciones móviles. Fundación Telefónica.
- Atlassian. (2023). *About us: Trello history, logos & customers / Trello*. Obtenido de Atlassian Trello: <https://trello.com/about>
- Azuma, R. (1997). *A Survey of Augmented Reality*. Obtenido de <https://www.cs.unc.edu/~azuma/ARpresence.pdf>
- B, G. (13 de marzo de 2025). *Qué es GitHub y cómo empezar a usarlo*. Obtenido de ES Tutoriales: <https://www.hostinger.com/es/tutoriales/que-es-github>
- Beck, K., & Andres, C. (2005). Extreme Programming Explained: Embrace change. En *Extreme Programming Explained*. Addison-Wesley.
- Booch, G., Maksimchuk, R., Engel, M., Young, B., Conallen, J., & Houston, K. (2007). *Object-Oriented Analysis and Design with Applications*. Addison-Wesley.
- Cloud, G. (2025). *Inteligencia Artificial (IA): una guía fácil de entender*. Obtenido de <https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419>
- Corporation, O. (2024). *MySQL Documentation*. Obtenido de MySQL: <https://dev.mysql.com/doc/>

- Cuadrado, G. C. (22 de julio de 2022). *OpenWebinars*. Obtenido de <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/>
- Curso para desarrollo de apps con React Native*. (s.f.). Obtenido de Código Facilito: <https://codigofacilito.com/articulos/que-es-react-native>
- Deloitte. (2019). *¿Qué es React Native? | Tecnología | Deloitte España*. Obtenido de Deloitte: <https://www.deloitte.com/es/es/services/consulting/blogs/todo-tecnologia/que-es-react-native.html>
- Developers, G. F. (2025). *Crea nuevas experiencias de realidad aumentada que combinen perfectamente con el mundo digital con el físico | ARCore | Google for Developers*. Obtenido de ARCore: <https://developers.google.com/ar?hl=es-419>
- DuBois, P. (2013). *MySQL*. Addison-Wesley Professional.
- Edward Laux, J. S. (2016). Obtenido de BodBot AI Entrenador Personal: <https://www.bodbot.com/>
- Flutter, s. (s.f.). *Flutter - Crea aplicaciones para cualquier pantalla*. Obtenido de <https://esflutter.dev/>
- Gabriel Maide, E., & Pacienza, J. (noviembre de 2020). *Metodologías de desarrollo de software aplicadas a proyectos de automatización de procesos*. Obtenido de Pistas Educativas: <https://pistaseducativas.celaya.tecnm.mx/index.php/pistas/article/viewFile/2381/1900>
- GeeksforGeeks. (11 de julio de 2025). *Incremental Process Model Software Engineering*. Obtenido de GeeksforGeeks: <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model/>
- Gironés, T. (2012). *Aplicaciones móviles: Conceptos y desarrollo*.
- Git. (s.f.). Obtenido de <http://git-scm.com/>
- Gjestvang, C. S. (s.f.). *Motives and barriers to initiation and sustained exercise adherence in new fitness club members: A prospective study*. Publicado en *Frontiers in Psychology*. Obtenido de <https://psycnet.apa.org/record/2020-64605-023>
- Global, S. (noviembre de 2025). Obtenido de Mobile Operating System Market Share Plurinational State of Bolivia | StatCounter Global Stats: <https://gs.statcounter.com/os-market-share/mobile/bolivia>

- Gómez, & Hernández. (2016). Mobile-D: Metodología ágil para el desarrollo de aplicaciones móviles. *Revista Orbis*.
- Gómez, A. A. (2015). *Comparación de dos tecnologías de desarrollo de aplicaciones móviles desde la perspectiva de los atributos de calidad*. Obtenido de ResearchGate: https://www.researchgate.net/figure/Figura-5-Arquitectura-de-Android_fig3_319975215
- Gómez, A. A. (2015). *Comparación de dos tecnologías de desarrollo de aplicaciones móviles desde la perspectiva de los atributos de calidad*. Universidad Nacional de Colombia: Bogotá.
- Google. (2023). *Android Developers – Official Documentation*. Obtenido de <https://developer.android.com/>
- Group, L. F. (2021). *Ejercicios en el Gimnasio*. Obtenido de https://play.google.com/store/apps/details?id=gymworkout.gym.gymlog.gymtrainer&hl=es_BO
- Guide, I. S. (12 de septiembre de 2023). *Modelo en espiral: modelo de reducción de riesgos para proyectos de software*. Obtenido de <https://www.ionos.mx/startupguide/productividad/modelo-en-espiral/>
- Gurnov, A., & Wrike (W.). (13 de noviembre de 2025). *Las 21 mejores herramientas de gestión de proyectos*. Obtenido de Wrike: <https://www.wrike.com/es/project-management-guide/faq/que-son-las-herramientas-de-gestion-de-proyectos/>
- Hayes, M., & Downie, A. (28 de noviembre de 2025). *Realidad aumentada*. IBM. Obtenido de IBM: <https://www.ibm.com/es-es/think/topics/augmented-reality>
- Huang, S. (2020). *NoSQL Databases: Concepts and Applications*. Cambridge: Morgan Kaufmann.
- IBM. (2025). *¿Qué son las redes neuronales?* Obtenido de <https://www.ibm.com/es-es/think/topics/neural-networks>
- IEEE. (1990). *IEEE Standard Glossary of Software Engineering Terminology*. Obtenido de Institute of Electrical and Electronics Engineers.
- Inc., A. (2025). *ARKit 6 - Augmented Reality - Apple Developer*.
- Ionic Documentation, F. (2023). *Ionic Framework*. Obtenido de Ionic Documentation: <https://ionicframework.com/docs>

- Isaac. (2025). *Firestore: Qué es y cómo funciona la plataforma de Google*. Obtenido de Mundobytes: <https://mundobytes.com/firebase-que-es/>
- Kolt, G., & Snyder-Mackler, L. (2007). *Terapias físicas en el deporte y el ejercicio*.
- Leyva, R., León, G., & García, F. (2016). Metodología Mobile-D para el desarrollo de aplicaciones móviles. *Revista Cubana de Ciencias Informáticas*, 10(3), 45–57. Obtenido de https://www.researchgate.net/figure/Figura-210-Metodologia-Mobile-D-Fuente-Leyva-et-al-2016_fig5_348295603
- López Mora, S. (noviembre de 2025). *Firestore: qué es, para qué sirve, funcionalidades y ventajas*. Obtenido de DIGITAL55: <https://digital55.com/blog/que-es-firebase-funcionalidades-ventajas-conclusiones/>
- Meier, R., & Lake, I. (2018). *Professional Android* (4.^a ed.).
- Microsoft. (2022). *Xamarin Documentation*. Obtenido de Microsoft Learn: <https://learn.microsoft.com/en-us/xamarin/>
- Mitchell, T. (1997). *Machine Learning*. New York: McGraw-Hill.
- MongoDB, I. (2024). *MongoDB Atlas documentation*. Obtenido de <https://www.mongodb.com/docs/atlas/>
- Navarro, F., Martínez, A., & Martínez, J. (2018). *Realidad Virtual y Realidad Aumentada*. Ra-Ma.
- Nielsen, J. (1993). *Usability Engineering*. Academic Press.
- Nystrom, K. (2021). *Kotlin Programming: The Big Nerd Ranch Guide*. Big Nerd Ranch. Pearson Education Store.
- Orientanet. (s.f.). *Orientanet*. Obtenido de ¿Qué es entrenamiento físico y tipos?: <https://www.orientanet.es/que-es-entrenamiento-fisico-y-tipos/>
- Pressman, R. S. (2010). *Software Engineering: A Practitioner's Approach*. Higher Education.
- Pykes, K. (2024). *Aprendizaje automático en Python: Tutorial de Scikit-Learn*. Obtenido de <https://www.datacamp.com/es/tutorial/machine-learning-python>
- React Native*. (2025). Obtenido de <https://reactnative.dev/docs/intro-react-native-components>
- Santander, B. (21 de diciembre de 2020). *Metodologías de desarrollo de software: ¿qué son?* Obtenido de Santander:

- <https://www.santanderopenacademy.com/es/blog/metodologias-desarrollo-software.html>
- Sanz, D. (2019). Obtenido de Introducción a Android: <https://dspace.itsjapon.edu.ec/jspui/bitstream/123456789/434/1/introduccion-android.pdf>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum*. Boston: Scrum.org.
- Services, A. W. (2025). Obtenido de ¿Qué es la IA? – Explicación de la inteligencia artificial – AWS: <https://aws.amazon.com/es/what-is/artificial-intelligence/>
- Sharma, M. (2018). *Productivity gains with modern IDEs: A review of IntelliJ-based environments*.
- Smith, J., & Darrow, M. (2019). *Gradle for Android Development*. O'Reilly Media.
- Sommerville, I. (2011). *Software Engineering*. Boston: Addison-Wesley.
- SQLite Consortium. (2024). *SQLite Home page*. Obtenido de SQLite.org: <https://sqlite.org/>
- Team. (2024). *Metodologías de desarrollo de software: guía para optimizar tu proyecto*. Obtenido de Blog de Desarrollo de Software, Testing e Inteligencia Artificial | Abstracta: <https://es.abstracta.us/blog/metodologias-de-desarrollo-de-software-guia-completa/>
- Tomás, J. (2019). *El gran libro de Android* (Vol. 7). Bogotá: Alfaomega.
- Viera, U. (s.f.). *Introducción a React Native: Crea Apps Móviles Nativas en iOS y Android*. Obtenido de urianviera.com: <https://urianviera.com/react-native/introduccion-a-react-native>
- Yépez, M. D. (2017). *Club Ensayos*. Obtenido de <https://www.clubensayos.com/Negocios/Proyecto-de-grado-Creaci%C3%B3n-de-un-centro-de/4574087.html>